

Week7:

Problem1:

Interface:

INotificationStrategy.cs

```
public interface INotificationStrategy
{
    string Type { get; }
    Task SendAsync(NotificationRequest request);
}
```

EmailNotificationStrategy.cs

```
public class EmailNotificationStrategy : INotificationStrategy
{
    public string Type => "Email";
    public async Task SendAsync(NotificationRequest request)
    {
        await Task.Delay(500);
        if (request.Recipient.Contains("fail"))
            throw new Exception("SMTP failure");
        Console.WriteLine($"Email sent to {request.Recipient}");
    }
}
```

SmsNotificationStrategy.cs

```
public class SmsNotificationStrategy : INotificationStrategy
{
    public string Type => "Sms";
    public async Task SendAsync(NotificationRequest request)
    {
        await Task.Delay(300);
        if (request.Recipient.Contains("fail"))
            throw new Exception("SMS timeout");
        Console.WriteLine($"SMS sent to {request.Recipient}");
    }
}
```

PushNotificationStrategy.cs

```
public class PushNotificationStrategy : INotificationStrategy
{
    public string Type => "Push";
    public async Task SendAsync(NotificationRequest request)
    {
        await Task.Delay(200);
        Console.WriteLine($"Push notification sent to {request.Recipient}");
    }
}
```

NotificationFactory.cs

```
public class NotificationFactory
{
    private readonly IEnumerable<INotificationStrategy> _strategies;
    public NotificationFactory(IEnumerable<INotificationStrategy> strategies)
    {
        _strategies = strategies;
    }
    public INotificationStrategy GetStrategy(string type)
    {
        var strategy = _strategies.FirstOrDefault(s => s.Type.Equals(type, StringComparison.OrdinalIgnoreCase));
    }
}
```

```

if (strategy == null)
throw new Exception("Invalid notification type");
return strategy;
}
}

```

NotificationService.cs

```

public class NotificationService
{
private readonly NotificationFactory _factory;
public NotificationService(NotificationFactory factory)
{
_factory = factory;
}
public async Task SendBulkAsync(List<NotificationRequest> requests)
{
var tasks = requests.Select(async request =>
{
var strategy = _factory.GetStrategy(request.Type);
try
{
await strategy.SendAsync(request);
}
catch (Exception)
{
try
{
await strategy.SendAsync(request);
}
catch (Exception ex)
{
Console.WriteLine($"Failed after retry: {ex.Message}");
}
}
});
await Task.WhenAll(tasks);
}
}

```

NotificationController.cs

```

[ApiController]
[Route("api/notifications")]
public class NotificationController : ControllerBase
{
private readonly NotificationService _service;
public NotificationController(NotificationService service)
{
_service = service;
}
[HttpPost("bulk")]
public async Task<ActionResult> SendBulk(List<NotificationRequest> requests)
{
await _service.SendBulkAsync(requests);
return Ok("Notifications processed");
}
}

```

NotificationRequest.cs

```

public class NotificationRequest
{
    public string Type { get; set; }
    public string Recipient { get; set; }
    public string Message { get; set; }
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddTransient<INotificationStrategy, EmailNotificationStrategy>();
builder.Services.AddTransient<INotificationStrategy, SmsNotificationStrategy>();
builder.Services.AddTransient<INotificationStrategy, PushNotificationStrategy>();
builder.Services.AddSingleton<NotificationFactory>();
builder.Services.AddTransient<NotificationService>();
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.Run();
*****

```

Problem2: THREAD-SAFE IN-MEMORY CACHE WITH EXPIRATION BUG

1.Models/CacheItem.cs

```

public class CacheItem
{
    public object Value { get; set; }
    public DateTime Expiry { get; set; }
}

```

2. Services/ICacheService.cs

```

public interface ICacheService
{
    void Set(string key, object value, TimeSpan expiry);
    object Get(string key);
}

```

3. Services/InMemoryCacheService.cs

```

using System.Collections.Concurrent;

```

```

public class InMemoryCacheService : ICacheService
{
    private readonly ConcurrentDictionary<string, CacheItem> _cache;

    public InMemoryCacheService(ConcurrentDictionary<string, CacheItem> cache)
    {
        _cache = cache;
    }

```

```

    public void Set(string key, object value, TimeSpan expiry)
    {
        var item = new CacheItem
        {
            Value = value,

```

```

    Expiry = DateTime.UtcNow.Add(expiry)
};
_cache[key] = item;
}

public object Get(string key)
{
    if (_cache.TryGetValue(key, out var item))
    {
        if (item.Expiry > DateTime.UtcNow)
        {
            return item.Value;
        }
        else
        {
            _cache.TryRemove(key, out _);
        }
    }
    return null;
}
}

```

4. Background/CleanupService.cs

```

using System.Collections.Concurrent;

public class CleanupService
{
    private readonly ConcurrentDictionary<string, CacheItem> _cache;

    public CleanupService(ConcurrentDictionary<string, CacheItem> cache)
    {
        _cache = cache;
    }

    public async Task StartAsync()
    {
        {
            while (true)
            {
                foreach (var key in _cache.Keys)
                {
                    {
                        if (_cache.TryGetValue(key, out var item))
                        {
                            if (item.Expiry <= DateTime.UtcNow)
                            {
                                _cache.TryRemove(key, out _);
                            }
                        }
                    }
                }
                await Task.Delay(5000);
            }
        }
    }
}

```

5. Program.cs

```

using System.Collections.Concurrent;

```

```

class Program
{
static async Task Main()
{
var cache = new ConcurrentDictionary<string, CacheItem>();
var service = new InMemoryCacheService(cache);
var cleanup = new CleanupService(cache);
_ = cleanup.StartAsync();
service.Set("key1", "Hello", TimeSpan.FromSeconds(10));
Console.WriteLine(service.Get("key1"));
await Task.Delay(15000);
Console.WriteLine(service.Get("key1"));
}
}
*****

```

Problem3: JWT Authentication System with Random Failures
JwtAuthApp

Controllers/AuthController.cs

```

using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly TokenService _tokenService;

    public AuthController(TokenService tokenService)
    {
        _tokenService = tokenService;
    }

    [HttpPost("login")]
    public IActionResult Login(LoginRequest request)
    {
        if (request.Username == "admin" && request.Password == "123")
        {
            var token = _tokenService.GenerateToken("1", "Admin");
            return Ok(new { token });
        }

        return Unauthorized();
    }
}

```

Controllers/TestController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/test")]
public class TestController : ControllerBase
{
    [Authorize]
    [HttpGet]
    public IActionResult Get()
    {
    }
}

```

```

    {
        return Ok("Protected data accessed");
    }
}

```

Models/LoginRequest.cs

```

public class LoginRequest
{
    public string Username { get; set; }
    public string Password { get; set; }
}

```

Services/TokenService.cs

```

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public class TokenService
{
    private readonly string _key = "THIS_IS_SUPER_SECRET_KEY_123456789012";

    public string GenerateToken(string userId, string role)
    {
        var claims = new[]
        {
            new Claim("UserId", userId),
            new Claim(ClaimTypes.Role, role)
        };

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_key));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(15),
            signingCredentials: creds
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

Program.cs

```

using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

var key = "THIS_IS_SUPER_SECRET_KEY_123456789012";

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

```

```

builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = Microsoft.OpenApi.Models.ParameterLocation.Header,
        Description = "Enter: Bearer {your token}"
    });

    options.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
    {
        {
            new Microsoft.OpenApi.Models.OpenApiSecurityScheme
            {
                Reference = new Microsoft.OpenApi.Models.OpenApiReference
                {
                    Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            new string[] {}
        }
    });
});

builder.Services.AddSingleton<TokenService>();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options =>
{
    options.RequireHttpsMetadata = false;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = false,
        ValidateAudience = false,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ClockSkew = TimeSpan.Zero,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(key))
    };
});

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();

```

OUTPUT

POST /api/auth/login

Request

```
{
  "username": "admin",
  "password": "123"
}
```

Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0:"
}
```

GET /api/test

Response

Protected data accessed

Problem4: RateLimiterApp

Controllers/TestController.cs

using Microsoft.AspNetCore.Mvc;

namespace RateLimiterApp.Controllers

```
{
    [ApiController]
    [Route("api/test")]
    public class TestController : ControllerBase
    {
        [HttpGet]
        public IActionResult Get()
        {
            return Ok("Request successful");
        }
    }
}
```

Services/IRateLimiter.cs

namespace RateLimiterApp.Services

```
{
    public interface IRateLimiter
    {
        bool IsAllowed(string key);
    }
}
```

Services/InMemoryRateLimiter.cs

using System.Collections.Concurrent;

namespace RateLimiterApp.Services

```
{
    public class InMemoryRateLimiter : IRateLimiter
```



```

{
    private readonly ConcurrentDictionary<string, List<DateTime>> _requests = new();

    private readonly int _limit = 5;
    private readonly TimeSpan _window = TimeSpan.FromSeconds(10);

    public bool IsAllowed(string key)
    {
        var now = DateTime.UtcNow;

        var timestamps = _requests.GetOrAdd(key, _ => new List<DateTime>());

        lock (timestamps)
        {
            timestamps.RemoveAll(t => t <= now - _window);

            if (timestamps.Count >= _limit)
            {
                return false;
            }

            timestamps.Add(now);
            return true;
        }
    }
}

```

Middleware/RateLimitingMiddleware.cs

```

using Microsoft.AspNetCore.Http;
using RateLimiterApp.Services;

namespace RateLimiterApp.Middleware
{
    public class RateLimitingMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly IRateLimiter _rateLimiter;

        public RateLimitingMiddleware(RequestDelegate next, IRateLimiter rateLimiter)
        {
            _next = next;
            _rateLimiter = rateLimiter;
        }

        public async Task InvokeAsync(HttpContext context)
        {
            var key = context.Connection.RemoteIpAddress?.ToString() ?? "unknown";

            if (!_rateLimiter.IsAllowed(key))
            {
                context.Response.StatusCode = 429;
                await context.Response.WriteAsync("Too Many Requests");
                return;
            }

            await _next(context);
        }
    }
}

```

```
    }  
  }  
}
```

Program.cs

```
using Microsoft.AspNetCore.Builder;  
using Microsoft.Extensions.DependencyInjection;  
using Microsoft.Extensions.Hosting;  
using RateLimiterApp.Services;  
using RateLimiterApp.Middleware;  
  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddSingleton<IRateLimiter, InMemoryRateLimiter>();  
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
  
var app = builder.Build();  
  
app.UseSwagger();  
app.UseSwaggerUI();  
  
app.UseMiddleware<RateLimitingMiddleware>();  
  
app.MapControllers();  
  
app.Run();
```

OUTPUT for

GET /api/test

```
Request 1 → Request successful  
Request 2 → Request successful  
Request 3 → Request successful  
Request 4 → Request successful  
Request 5 → Request successful  
Request 6 → Too Many Requests (429)  
*****
```

Problem5:

DeadlockApp

Program.cs

```
using System;  
using System.Threading;  
using System.Threading.Tasks;  
  
namespace DeadlockApp  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Starting Deadlock Demo...");  
        }  
    }  
}
```

```

var resource1 = new object();
var resource2 = new object();

// Deadlock scenario: two threads locking resources in opposite order
Task t1 = Task.Run(() => DeadlockMethod1(resource1, resource2));
Task t2 = Task.Run(() => DeadlockMethod2(resource1, resource2));

// This will hang because both threads wait forever
Task.WaitAll(t1, t2);

Console.WriteLine("This line will never execute in deadlock case");
}

// Thread 1 locks Resource1 first, then Resource2
static void DeadlockMethod1(object res1, object res2)
{
    lock (res1)
    {
        Console.WriteLine("Thread 1 locked Resource 1");
        Thread.Sleep(1000); // simulate delay

        lock (res2)
        {
            Console.WriteLine("Thread 1 locked Resource 2");
        }
    }
}

// Thread 2 locks Resource2 first, then Resource1
static void DeadlockMethod2(object res1, object res2)
{
    lock (res2)
    {
        Console.WriteLine("Thread 2 locked Resource 2");
        Thread.Sleep(1000); // simulate delay

        lock (res1)
        {
            Console.WriteLine("Thread 2 locked Resource 1");
        }
    }
}

// FIX 1: Lock ordering (both threads follow same order)
static void SafeMethod(object res1, object res2)
{
    lock (res1)
    {
        Console.WriteLine("Thread locked Resource 1");
        Thread.Sleep(500);

        lock (res2)
        {
            Console.WriteLine("Thread locked Resource 2");
        }
    }
}

```

```

// FIX 2: Timeout mechanism to avoid infinite waiting
static void TimeoutMethod(object res1, object res2)
{
    if (Monitor.TryEnter(res1, TimeSpan.FromSeconds(2)))
    {
        try
        {
            Console.WriteLine("Locked Resource 1");

            Thread.Sleep(500);

            if (Monitor.TryEnter(res2, TimeSpan.FromSeconds(2)))
            {
                try
                {
                    Console.WriteLine("Locked Resource 2");
                }
                finally
                {
                    Monitor.Exit(res2);
                }
            }
            else
            {
                Console.WriteLine("Timeout while waiting for Resource 2");
            }
        }
        finally
        {
            Monitor.Exit(res1);
        }
    }
    else
    {
        Console.WriteLine("Timeout while waiting for Resource 1");
    }
}

// Deadlock detection using timeout check
static async Task DetectDeadlock(Task t1, Task t2)
{
    var completed = await Task.WhenAny(Task.WhenAll(t1, t2), Task.Delay(3000));

    if (completed != Task.WhenAll(t1, t2))
    {
        Console.WriteLine("Deadlock detected!");
    }
}
}

```

OUTPUT

Deadlock Scenario

Starting Deadlock Demo...

Thread 1 locked Resource 1

Thread 2 locked Resource 2

(program stuck forever)

Fixed using Lock Ordering

Thread locked Resource 1

Thread locked Resource 2

Execution completed successfully

Fixed using Timeout

Locked Resource 1

Timeout while waiting for Resource 2

Problem6: CustomHashMapApp

Models/HashNode.cs

using System;

namespace CustomHashMapApp.Models

```
{
    // This node represents each entry in hashmap (used for collision chaining)
    public class HashNode<K, V>
    {
        public K Key { get; set; }
        public V Value { get; set; }
        public HashNode<K, V> Next { get; set; }

        public HashNode(K key, V value)
        {
            Key = key;
            Value = value;
            Next = null;
        }
    }
}
```

Services/CustomHashMap.cs

using System;

using CustomHashMapApp.Models;

namespace CustomHashMapApp.Services

```
{
    // Custom HashMap implementation using array + linked list (chaining)
    public class CustomHashMap<K, V>
    {
        private HashNode<K, V>[] buckets;
        private int capacity;
        private int size;

        // Load factor threshold to trigger resizing
        private const double LOAD_FACTOR = 0.75;

        public CustomHashMap(int initialCapacity = 4)
        {
            capacity = initialCapacity;
            buckets = new HashNode<K, V>[capacity];
        }
    }
}
```

```

        size = 0;
    }

    // Hash function to convert key into index
    private int GetIndex(K key)
    {
        int hash = key.GetHashCode();
        return Math.Abs(hash) % capacity;
    }

    // Add or update key-value
    public void Put(K key, V value)
    {
        int index = GetIndex(key);

        var head = buckets[index];

        // If key already exists, update value
        while (head != null)
        {
            if (head.Key.Equals(key))
            {
                head.Value = value;
                return;
            }
            head = head.Next;
        }

        // Insert new node at beginning (chaining)
        var newNode = new HashNode<K, V>(key, value);
        newNode.Next = buckets[index];
        buckets[index] = newNode;

        size++;

        // Resize if load factor exceeded
        if ((double)size / capacity > LOAD_FACTOR)
        {
            Resize();
        }
    }

    // Get value by key
    public V Get(K key)
    {
        int index = GetIndex(key);

        var head = buckets[index];

        while (head != null)
        {
            if (head.Key.Equals(key))
            {
                return head.Value;
            }
            head = head.Next;
        }
    }

```

```

        return default(V); // return default if not found
    }

    // Remove key
    public void Remove(K key)
    {
        int index = GetIndex(key);

        var head = buckets[index];
        HashNode<K, V> prev = null;

        while (head != null)
        {
            if (head.Key.Equals(key))
            {
                // remove node from chain
                if (prev == null)
                    buckets[index] = head.Next;
                else
                    prev.Next = head.Next;

                size--;
                return;
            }

            prev = head;
            head = head.Next;
        }
    }

    // Resize hashmap when load factor exceeds threshold
    private void Resize()
    {
        int newCapacity = capacity * 2;
        var newBuckets = new HashNode<K, V>[newCapacity];

        // Rehash all elements into new bucket array
        foreach (var node in buckets)
        {
            var current = node;
            while (current != null)
            {
                int newIndex = Math.Abs(current.Key.GetHashCode()) % newCapacity;

                var temp = current.Next;

                current.Next = newBuckets[newIndex];
                newBuckets[newIndex] = current;

                current = temp;
            }
        }

        buckets = newBuckets;
        capacity = newCapacity;
    }
}

```

Program.cs

```
using System;
using CustomHashMapApp.Services;

namespace CustomHashMapApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Custom HashMap Demo");

            var map = new CustomHashMap<string, int>();

            // Adding elements
            map.Put("A", 1);
            map.Put("B", 2);
            map.Put("C", 3);

            // Getting values
            Console.WriteLine("Value for A: " + map.Get("A"));
            Console.WriteLine("Value for B: " + map.Get("B"));

            // Updating existing key
            map.Put("A", 100);
            Console.WriteLine("Updated A: " + map.Get("A"));

            // Removing key
            map.Remove("B");
            Console.WriteLine("After removal B: " + map.Get("B"));

            Console.WriteLine("Execution Completed");
        }
    }
}
```

OUTPUT

Custom HashMap Demo

Value for A: 1

Value for B: 2

Updated A: 100

After removal B: 0

Execution Completed

Problem7: OrderSystemApp

Models/Order.cs

```
namespace OrderSystemApp.Models
{
    // This model represents an order in system
    public class Order
    {
        public string Id { get; set; }
        public string Product { get; set; }
    }
}
```



```

        public int Quantity { get; set; }
    }
}

```

Repositories/IOrderRepository.cs

```

using OrderSystemApp.Models;

namespace OrderSystemApp.Repositories
{
    // Repository interface to abstract data storage
    public interface IOrderRepository
    {
        void Add(Order order);
        Order Get(string id);
    }
}

```

Repositories/InMemoryOrderRepository.cs

```

using System.Collections.Concurrent;
using OrderSystemApp.Models;

namespace OrderSystemApp.Repositories
{
    // Thread-safe in-memory storage using ConcurrentDictionary
    public class InMemoryOrderRepository : IOrderRepository
    {
        private readonly ConcurrentDictionary<string, Order> _db = new();

        public void Add(Order order)
        {
            _db[order.Id] = order;
        }

        public Order Get(string id)
        {
            _db.TryGetValue(id, out var order);
            return order;
        }
    }
}

```

Cache/ICacheService.cs

```

namespace OrderSystemApp.Cache
{
    // Cache abstraction
    public interface ICacheService
    {
        void Set(string key, object value);
        object Get(string key);
    }
}

```

Cache/MemoryCacheService.cs

```

using System.Collections.Concurrent;

```

```

namespace OrderSystemApp.Cache
{
    // Simple in-memory cache to improve read performance
    public class MemoryCacheService : ICacheService
    {
        private readonly ConcurrentDictionary<string, object> _cache = new();

        public void Set(string key, object value)
        {
            _cache[key] = value;
        }

        public object Get(string key)
        {
            _cache.TryGetValue(key, out var value);
            return value;
        }
    }
}

```

Notifications/INotificationService.cs

```
using OrderSystemApp.Models;
```

```

namespace OrderSystemApp.Notifications
{
    // Strategy pattern for notifications
    public interface INotificationService
    {
        void Send(Order order);
    }
}

```

Notifications/EmailNotification.cs

```
using OrderSystemApp.Models;
```

```

namespace OrderSystemApp.Notifications
{
    // Simulates email notification
    public class EmailNotification : INotificationService
    {
        public void Send(Order order)
        {
            Console.WriteLine($"Email sent for Order {order.Id}");
        }
    }
}

```

Services/IOrderService.cs

```
using OrderSystemApp.Models;
```

```

namespace OrderSystemApp.Services
{
    // Business logic interface
    public interface IOrderService

```

```

    {
        void PlaceOrder(Order order);
        Order GetOrder(string id);
    }
}

```

Services/OrderService.cs

```

using OrderSystemApp.Models;
using OrderSystemApp.Repositories;
using OrderSystemApp.Cache;
using OrderSystemApp.Notifications;

namespace OrderSystemApp.Services
{
    // Core logic handling order processing
    public class OrderService : IOrderService
    {
        private readonly IOrderRepository _repo;
        private readonly ICacheService _cache;
        private readonly INotificationService _notification;

        public OrderService(IOrderRepository repo, ICacheService cache, INotificationService notification)
        {
            _repo = repo;
            _cache = cache;
            _notification = notification;
        }

        public void PlaceOrder(Order order)
        {
            // Save order in repository
            _repo.Add(order);

            // Store in cache for fast retrieval
            _cache.Set(order.Id, order);

            // Send notification (failure should not affect order)
            try
            {
                _notification.Send(order);
            }
            catch
            {
                Console.WriteLine("Notification failed but order is saved");
            }
        }

        public Order GetOrder(string id)
        {
            // Check cache first (fast)
            var cached = _cache.Get(id) as Order;
            if (cached != null)
                return cached;

            // Fetch from repository if not in cache
            var order = _repo.Get(id);

```

```

        if (order != null)
            _cache.Set(id, order);

        return order;
    }
}

```

Controllers/OrderController.cs

```

using Microsoft.AspNetCore.Mvc;
using OrderSystemApp.Services;
using OrderSystemApp.Models;

namespace OrderSystemApp.Controllers
{
    [ApiController]
    [Route("api/order")]
    public class OrderController : ControllerBase
    {
        private readonly IOrderService _service;

        public OrderController(IOrderService service)
        {
            _service = service;
        }

        [HttpPost]
        public IActionResult PlaceOrder(Order order)
        {
            _service.PlaceOrder(order);
            return Ok("Order placed");
        }

        [HttpGet("{id}")]
        public IActionResult GetOrder(string id)
        {
            var order = _service.GetOrder(id);

            if (order == null)
                return NotFound();

            return Ok(order);
        }
    }
}

```

Middleware/RateLimitingMiddleware.cs

```

using Microsoft.AspNetCore.Http;
using System.Collections.Concurrent;

namespace OrderSystemApp.Middleware
{
    // Simple rate limiting to control high traffic
    public class RateLimitingMiddleware
    {
        private readonly RequestDelegate _next;
    }
}

```

```

private static ConcurrentDictionary<string, int> _requests = new();

public RateLimitingMiddleware(RequestDelegate next)
{
    _next = next;
}

public async Task InvokeAsync(HttpContext context)
{
    var key = context.Connection.RemoteIpAddress?.ToString() ?? "unknown";

    _requests.AddOrUpdate(key, 1, (k, v) => v + 1);

    if (_requests[key] > 5)
    {
        context.Response.StatusCode = 429;
        await context.Response.WriteAsync("Too many requests");
        return;
    }

    await _next(context);
}
}
}

```

Program.cs

```

using OrderSystemApp.Services;
using OrderSystemApp.Repositories;
using OrderSystemApp.Cache;
using OrderSystemApp.Notifications;
using OrderSystemApp.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// Dependency Injection setup
builder.Services.AddSingleton<IOrderRepository, InMemoryOrderRepository>();
builder.Services.AddSingleton<ICacheService, MemoryCacheService>();
builder.Services.AddSingleton<INotificationService, EmailNotification>();
builder.Services.AddScoped<IOrderService, OrderService>();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseMiddleware<RateLimitingMiddleware>();

app.MapControllers();

app.Run();

```

OUTPUT

POST /api/order

Request:

```
{
  "id": "1",
  "product": "Book",
  "quantity": 2
}
```

Response:

Order placed

In the Console:

Email sent for Order 1

GET /api/order/1

Response:

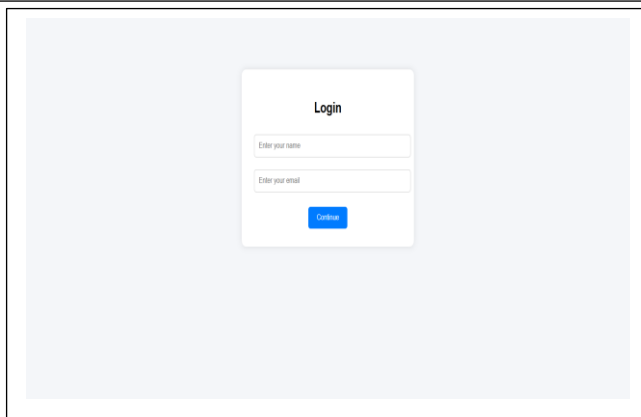
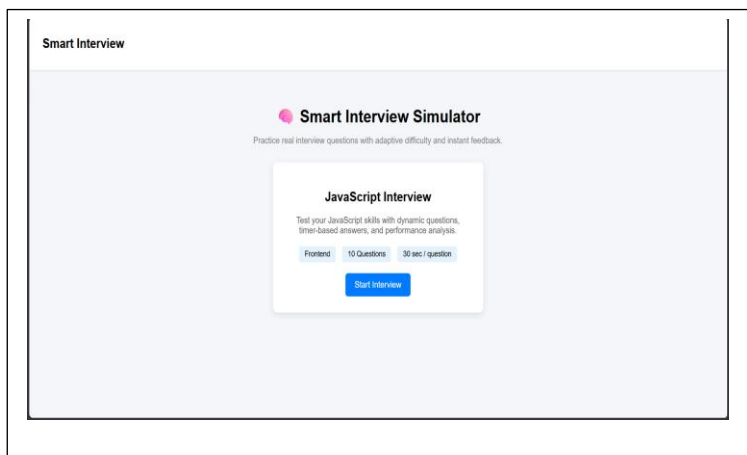
```
{
  "id": "1",
  "product": "Book",
  "quantity": 2
}
```

After multiple requests:

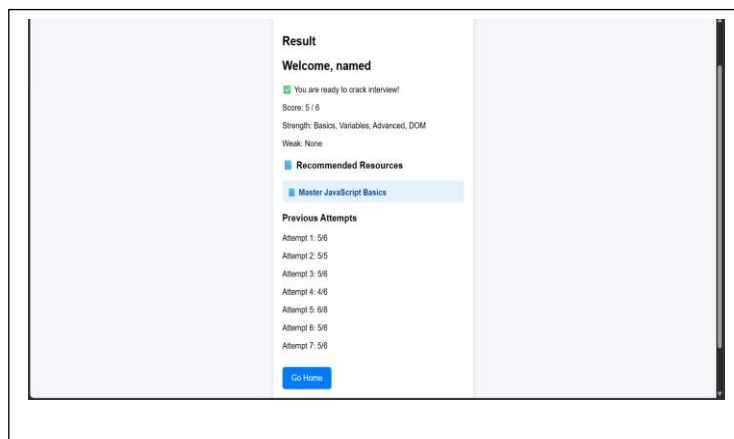
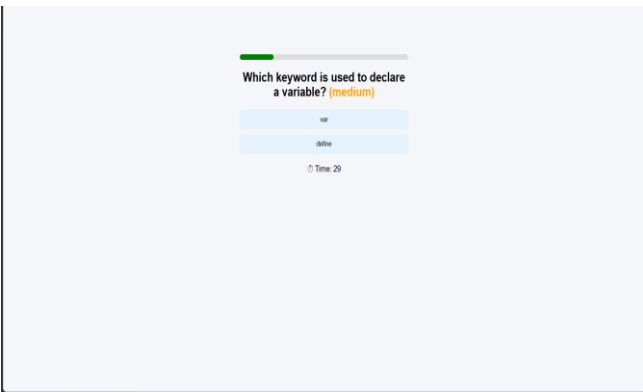
429 Too many requests

PROJECT (EXAM) IMAGES:

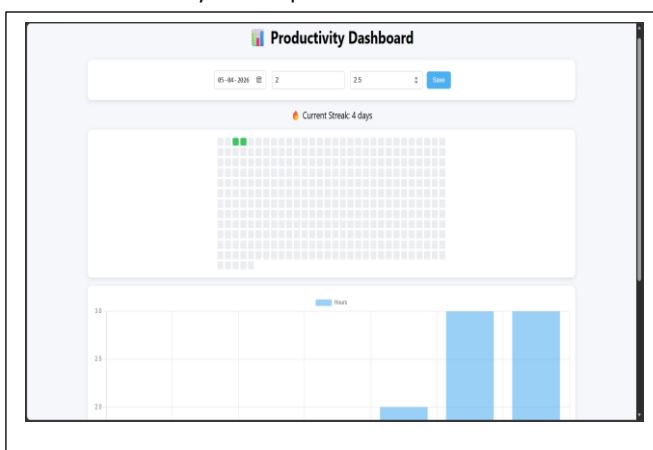
SMARTINTERVIEW:



In the interview page, the main logic is that questions are loaded dynamically using a decision tree concept. If the user answers correctly, it moves to a harder question, and if the answer is wrong, it goes to an easier question



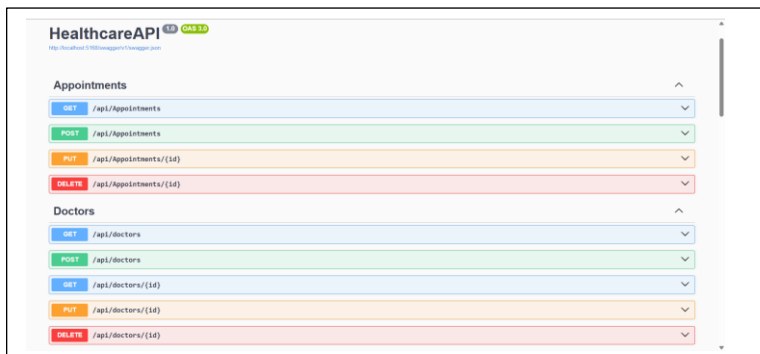
Personal Productivity Heatmap Dashboard



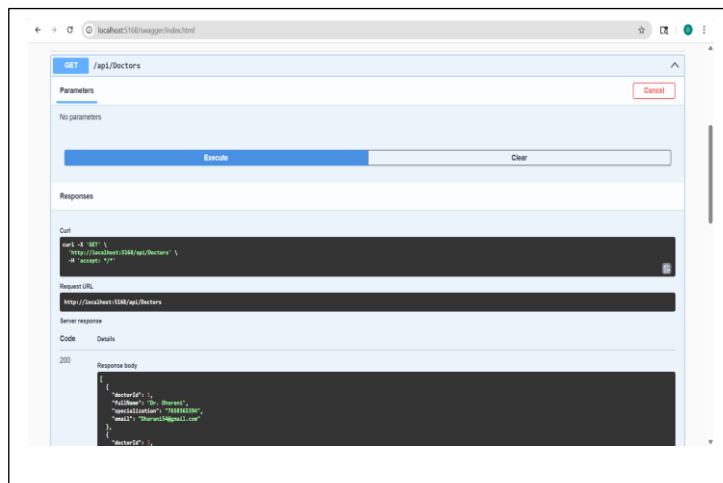
This project visualizes daily productivity in a GitHub-style heatmap, where color intensity dynamically reflects study hours and completed tasks

HealthCareAPI:

I created models → Entity Framework Manages tables →
controllers created APIs → APIs fetch data from database →

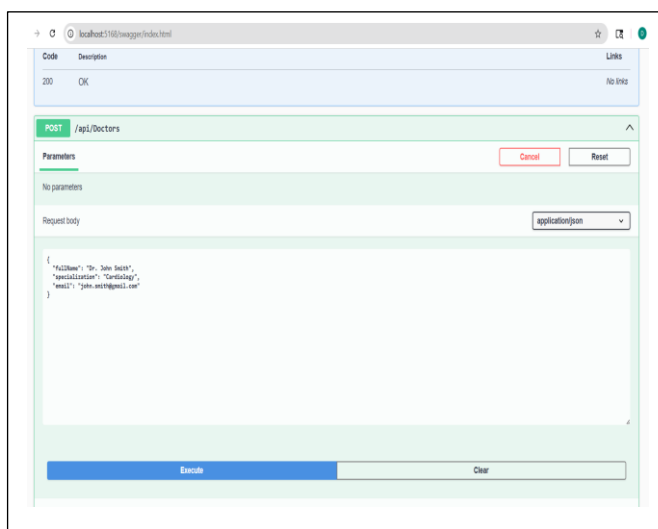


Doctor API GET:



I created one more record using PUT

DOCTOR POST Http Method:



From This Dashboard I can directly navigate to Swagger API

Healthcare Dashboard

PatientsDoctors

Search...

ID	Name	Email	Phone	Created	Actions	
2	DivyaSony	Divya714@gmail.com	7894336758	01 Jan 1	EditDelete	
3	Kaulehya	Kauleha54@gmail.com	9785573984	01 Jan 1	EditDelete	

Open API (Swagger)